Plan

Plan

Four lectures on the MatterMost paper:

- The theory behind the MatterMost paper (three lectures)
- The main idea of the MatterMost paper (one lecture)

Plan

Four lectures on the MatterMost paper:

- The theory behind the MatterMost paper (three lectures)
- The main idea of the MatterMost paper (one lecture)

Today:

- The theory behind the MatterMost paper 2-3

Wrap up MM1 lecture

Wrap up MM1 lecture

- We have seen two examples of (finite horizon) SDPs. In both examples . . .

Wrap up MM1 lecture

- We have seen two examples of (finite horizon) SDPs. In both examples ...
- ... decisions have to be made sequentially,

Wrap up MM1 lecture

- We have seen two examples of (finite horizon) SDPs. In both examples ...
- ... decisions have to be made sequentially,
- ... the consequences of these decisions are uncertain.

Wrap up MM1 lecture

- We have seen two examples of (finite horizon) SDPs. In both examples ...
- ... decisions have to be made sequentially,
- ... the consequences of these decisions are uncertain.
- ... uncertainties can be of different kinds,

Wrap up MM1 lecture

- We have seen two examples of (finite horizon) SDPs. In both examples ...
- ... decisions have to be made sequentially,
- ... the consequences of these decisions are uncertain.
- ... uncertainties can be of different kinds,
- ... making a good decision at a given step requires accounting for all its immediate possible consequences and for all possible consequences of future decisions.

Responsibility under uncertainty: which (climate) decisions matter most?

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).
- The theory of SDPs can be applied to measure if decisions matter a lot or little.

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).
- The theory of SDPs can be applied to measure if decisions matter a lot or little.
- Caveat: responsibility attribution is a large field of research!

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).
- The theory of SDPs can be applied to measure if decisions matter a lot or little.
- Caveat: responsibility attribution is a large field of research!
- Caveat: important distinction are between individual and collective responsibility and between ex-ante and ex-post responsibility.

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).
- The theory of SDPs can be applied to measure if decisions matter a lot or little.
- Caveat: responsibility attribution is a large field of research!
- Caveat: important distinction are between individual and collective responsibility and between ex-ante and ex-post responsibility.

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).
- The theory of SDPs can be applied to measure if decisions matter a lot or little.
- Caveat: responsibility attribution is a large field of research!
- Caveat: important distinction are between individual and collective responsibility and between ex-ante and ex-post responsibility.
- Responsibility is also closely related to causality.

Responsibility under uncertainty: which (climate) decisions matter most?

- What has the MM paper to do with SDPs?
- The basic idea is in the title: responsibility is high (low) if decisions matter a lot (little).
- The theory of SDPs can be applied to measure if decisions matter a lot or little.
- Caveat: responsibility attribution is a large field of research!
- Caveat: important distinction are between individual and collective responsibility and between ex-ante and ex-post responsibility.
- Responsibility is also closely related to causality.
- Ex-ante (type, general) and ex-post (actual, specific) causality are discussed in pages 1-5 of Joseph Y. Halpern. Actual Causality (2016).

The theory behind the paper 2

The theory in a nutshell

- The theory (library) consists of two sets of software components.

The theory in a nutshell

- The theory (library) consists of two sets of software components.
- A set of specification components. These are partial definitions that describe an abstract SDP.

The theory in a nutshell

- The theory (library) consists of two sets of software components.
- A set of specification components. These are partial definitions that describe an abstract SDP.
- A set of solution components. These are generic data types and functions for computing a sequence of optimal policies via verified backward induction.

The theory in a nutshell

- The theory (library) consists of two sets of software components.
- A set of specification components. These are partial definitions that describe an abstract SDP.
- A set of solution components. These are generic data types and functions for computing a sequence of optimal policies via verified backward induction.
- The theory is applied (to a specific SDP) by completing the partial definitions and calling the solution components.

Specification

Specification: uncertainty

- Uncertainty is represented by an uncertainty monad M.

Specification: uncertainty

- Uncertainty is represented by an uncertainty monad M.
- A monad is functor (remember OUU!) ...

$$
\begin{aligned}
M &: Set \to Set \\
fmap_M &: \{A\ B : Set\} \to (A \to B) \to M\ A \to M\ B
\end{aligned}
$$

Specification: uncertainty

- Uncertainty is represented by an uncertainty monad M.
- A monad is functor (remember OUU!) ...

$$
\begin{aligned}
M &: Set \to Set \\
fmap_M &: \{A\ B : Set\} \to (A \to B) \to M\ A \to M\ B
\end{aligned}
$$

- ...with a *return* (*pure*) η_M and a *join* μ_M:

$$
\begin{aligned}
\eta_M &: \{A : Set\} \to A \to M\ A \\
\mu_M &: \{A : Set\} \to M\ (M\ A) \to M\ A
\end{aligned}
$$

Specification: uncertainty

- These allow one to define a *bind* operation ...

$$_\gg\!=_M_ \;:\; \{B\ C\ :\ Set\} \to M\ B \to (B \to M\ C) \to M\ C$$
$$mb \gg\!=_M f \;=\; \mu_M\ (fmap_M\ f\ mb)$$

Specification: uncertainty

- These allow one to define a *bind* operation ...

$$_\gg\!=_M_ : \quad \{B\ C : Set\} \to M\ B \to (B \to M\ C) \to M\ C$$
$$mb \gg\!=_M f = \mu_M\ (fmap_M\ f\ mb)$$

- ... and a (Kleisli) composition:

$$_>\!=\!>_M_ : \{A\ B\ C : Set\} \to (A \to M\ B) \to (B \to M\ C) \to (A \to M\ C)$$
$$(f >\!=\!>_M g)\ a = (f\ a) \gg\!=_M g$$

Specification: uncertainty

- These allow one to define a *bind* operation ...

$$_\gg\!=_M_ : \quad \{B\ C : Set\} \to M\ B \to (B \to M\ C) \to M\ C$$
$$mb \gg\!=_M f = \mu_M\ (fmap_M\ f\ mb)$$

- ... and a (Kleisli) composition:

$$_>\!=\!>_M_ : \{A\ B\ C : Set\} \to (A \to M\ B) \to (B \to M\ C) \to (A \to M\ C)$$
$$(f >\!=\!>_M g)\ a = (f\ a) \gg\!=_M g$$

Exercise 5.1

Make sure that you understand the definitions of $_\gg\!=_M_$ and $_>\!=\!>_M_$. What are the types of $fmap_M\ f\ mb$ and $f\ a$?

Specification: uncertainty, number of decision steps

- These allow one to define a *bind* operation ...

$$_\gg\!=_M_ : \quad \{B\ C : Set\} \to M\ B \to (B \to M\ C) \to M\ C$$
$$mb \gg\!=_M f = \mu_M\ (fmap_M\ f\ mb)$$

- ... and a (Kleisli) composition:

$$_>\!=\!>_M_ : \{A\ B\ C : Set\} \to (A \to M\ B) \to (B \to M\ C) \to (A \to M\ C)$$
$$(f >\!=\!>_M g)\ a = (f\ a) \gg\!=_M g$$

Exercise 5.2

Define *fmap*, η and μ for the identity functor ($Id\ A = A$) and for the *List* functor. What are $_\gg\!=_$ and $_>\!=\!>_$ for these functors? (remember the definition of *bind* in OUU3)

Specification: decision steps

- A finite horizon SDP is defined over a finite number of decision steps.

Specification: decision steps

- A finite horizon SDP is defined over a finite number of decision steps.
- At decision step $t : \mathbb{N}$, we have already taken t decisions.

Specification: states, controls, transition function

- The set of states of the problem can be different at different decision steps:

$$X : \mathbb{N} \to Set$$

Specification: states, controls, transition function

- The set of states of the problem can be different at different decision steps:

$$X : \mathbb{N} \to Set$$

- The set of controls can be different at different decision steps and in different states:

$$Y : (t : \mathbb{N}) \to X\ t \to Set$$

Specification: states, controls, transition function

- The set of states of the problem can be different at different decision steps:

$$X : \mathbb{N} \to Set$$

- The set of controls can be different at different decision steps and in different states:

$$Y : (t : \mathbb{N}) \to X\ t \to Set$$

- Selecting a control $y : Y\ t\ x$ in $x : X\ t$ yields an M-structure of possible next states:

$$next : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to M\ (X\ (suc\ t))$$

Specification: states, controls, transition function

- The set of states of the problem can be different at different decision steps:

$$X : \mathbb{N} \to Set$$

- The set of controls can be different at different decision steps and in different states:

$$Y : (t : \mathbb{N}) \to X\ t \to Set$$

- Selecting a control $y : Y\ t\ x$ in $x : X\ t$ yields an M-structure of possible next states:

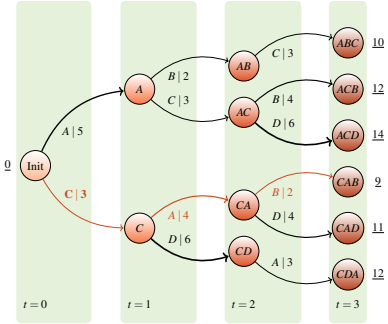
$$next : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to M\ (X\ (suc\ t))$$

Exercise 5.3

Compare *next* to *NonDetSys* from OUU3. What are the differences, what are the similarities? What would be needed to compute the flow and the trajectories of *next*?

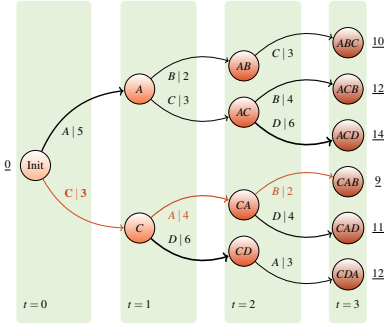
Specification: states, controls, transition function

- *next* describes a layered decision network with states in the nodes and controls in the edges:



Specification: states, controls, transition function

- *next* describes a layered decision network with states in the nodes and controls in the edges:



Exercise 5.4

The picture represents a deterministic system. Why? What would be the main difference in a picture of a non-deterministic system? Draw a non-deterministic decision network for a single step.

Specification: values, reward function

- Each decision step yields a reward in a value set Val:

$$Val \ : \ Set$$

Specification: values, reward function

- Each decision step yields a reward in a value set Val:

$$Val \ : \ Set$$

- As in OUU, we require Val to be a total preorder:

$$\begin{aligned}
\leqslant \quad &: \ Val \ \rightarrow \ Val \ \rightarrow \ Set \\
refl_{\leqslant} \quad &: \ (x \ : \ Val) \ \rightarrow \ x \ \leqslant \ x \\
trans_{\leqslant} \ &: \ (x \ y \ z \ : \ Val) \ \rightarrow \ x \ \leqslant \ y \ \rightarrow \ y \ \leqslant \ z \ \rightarrow \ x \ \leqslant \ z \\
total_{\leqslant} \ &: \ (x \ y \ : \ Val) \ \rightarrow \ Either \ (x \ \leqslant \ y) \ (y \ \leqslant \ x)
\end{aligned}$$

Specification: values, reward function

- Each decision step yields a reward in a value set *Val*:

$$Val \;:\; Set$$

- As in OUU, we require *Val* to be a total preorder:

$$\begin{aligned}
\leqslant &: Val \to Val \to Set \\
refl_{\leqslant} &: (x : Val) \to x \leqslant x \\
trans_{\leqslant} &: (x\ y\ z : Val) \to x \leqslant y \to y \leqslant z \to x \leqslant z \\
total_{\leqslant} &: (x\ y : Val) \to Either\ (x \leqslant y)\ (y \leqslant x)
\end{aligned}$$

- We will also need *Val* to have a reference "zero" element and an "addition":

$$\begin{aligned}
0_{Val} &: Val \\
_ < + > _ &: Val \to Val \to Val
\end{aligned}$$

Specification: reward function, solving a SDP

- Different combinations of current state, control and next state can lead to different rewards:

$$reward \ : \ (t \ : \ \mathbb{N}) \ \rightarrow \ (x \ : \ X \ t) \ \rightarrow \ Y \ t \ x \ \rightarrow \ X \ (suc \ t) \ \rightarrow \ Val$$

Specification: reward function, solving a SDP

- Different combinations of current state, control and next state can lead to different rewards:

$$\mathit{reward} : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to X\ (\mathit{suc}\ t) \to \mathit{Val}$$

- Solving a SDP means finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.

Specification: reward function, solving a SDP

- Different combinations of current state, control and next state can lead to different rewards:

$$\mathit{reward} : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to X\ (\mathit{suc}\ t) \to \mathit{Val}$$

- Solving a SDP means finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- A policy is a function that maps states to controls.

Specification: reward function, solving a SDP

- Different combinations of current state, control and next state can lead to different rewards:

$$\mathit{reward} : (t : \mathbb{N}) \to (x : X\ t) \to Y\ t\ x \to X\ (\mathit{suc}\ t) \to \mathit{Val}$$

- Solving a SDP means finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- A policy is a function that maps states to controls.

Exercise 5.5

Make sure that you understand what it means to solve an SDP.

Specification: reward function, solving a SDP

- Different combinations of current state, control and next state can lead to different rewards:

$$\mathit{reward} \;:\; (t \;:\; \mathbb{N}) \;\to\; (x \;:\; X\ t) \;\to\; Y\ t\ x \;\to\; X\ (\mathit{suc}\ t) \;\to\; \mathit{Val}$$

- Solving a SDP means finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- A policy is a function that maps states to controls.

Exercise 5.5

Make sure that you understand what it means to solve an SDP.

Exercise 5.6

Define M, X, Y and *next* for the generation dilemma. What could be *Val* and *reward* for this problem?

Solution

Solution: policies, policy sequences

- ...finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.

Solution: policies, policy sequences

- ...finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- We need to formulate the problem precisely.

Solution: policies, policy sequences

- ...finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- We need to formulate the problem precisely.
- We start with policies and policy sequences:

Solution: policies, policy sequences

- ...finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- We need to formulate the problem precisely.
- We start with policies and policy sequences:

$$
\begin{aligned}
\mathit{Policy} &: (t : \mathbb{N}) \to \mathit{Set} \\
\mathit{Policy}\ t &= (x : X\ t) \to Y\ t\ x
\end{aligned}
$$

Solution: policies, policy sequences

- ...finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- We need to formulate the problem precisely.
- We start with policies and policy sequences:

$$
\begin{aligned}
&\mathit{Policy} \quad : \quad (t : \mathbb{N}) \to \mathit{Set} \\
&\mathit{Policy}\ t \ = \ (x : X\ t) \to Y\ t\ x
\end{aligned}
$$

$$
\begin{aligned}
&\textbf{data}\ \mathit{PolicySeq} : (t\ n : \mathbb{N}) \to \mathit{Set}\ \textbf{where} \\
&\quad \mathit{Nil} \ : \{t : \mathbb{N}\} \quad \to \mathit{PolicySeq}\ t\ \mathit{zero} \\
&\quad _::_ : \{t\ n : \mathbb{N}\} \to \mathit{Policy}\ t \to \mathit{PolicySeq}\ (\mathit{suc}\ t)\ n \to \mathit{PolicySeq}\ t\ (\mathit{suc}\ n)
\end{aligned}
$$

Solution: policies, policy sequences

- ...finding a sequence of policies that maximizes a measure of the $\oplus$-sum of the rewards along all possible trajectories.
- We need to formulate the problem precisely.
- We start with policies and policy sequences:

$$
\begin{aligned}
&\mathit{Policy} \quad : \quad (t : \mathbb{N}) \to \mathit{Set} \\
&\mathit{Policy}\ t \ = \ (x : X\ t) \to Y\ t\ x
\end{aligned}
$$

$$
\begin{aligned}
&\textbf{data}\ \mathit{PolicySeq} : (t\ n : \mathbb{N}) \to \mathit{Set}\ \textbf{where} \\
&\quad \mathit{Nil} \ : \{t : \mathbb{N}\} \quad \to \mathit{PolicySeq}\ t\ \mathit{zero} \\
&\quad _::_ : \{t\ n : \mathbb{N}\} \to \mathit{Policy}\ t \to \mathit{PolicySeq}\ (\mathit{suc}\ t)\ n \to \mathit{PolicySeq}\ t\ (\mathit{suc}\ n)
\end{aligned}
$$

Exercise 5.7

Explain the $(\mathit{suc}\ t)\ n$ - $t\ (\mathit{suc}\ n)$ pattern in the definition of *PolicySeq*.